

Path: Cyber Security 101 > Module 2: Linux Fundamentals > Room 2: Linux Fundamentals Part 2

<https://tryhackme.com/room/linuxfundamentalspart2>

Linux Fundamentals Part 2:

Continue your learning Linux journey with part two. You will be learning how to log in to a Linux machine using SSH, how to advance your commands, file system interaction.

>> Task 1: Introduction

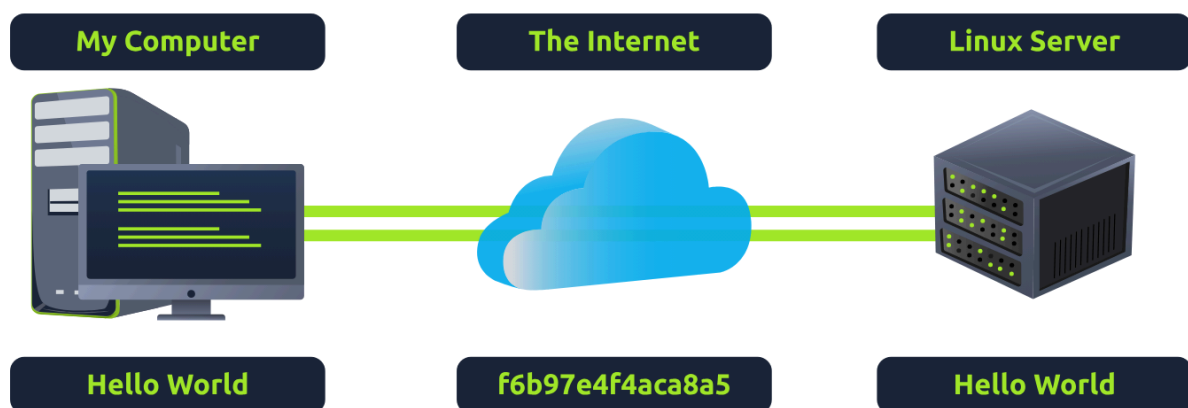
Welcome to the second part of the reworked **Linux Fundamentals** series. We'll be applying our knowledge from the first installment in this series.

>> Task 2: Accessing Your Linux Machine Using SSH (Deploy)

What is SSH & how Does it Work?

Secure Shell (SSH): Secure Shell (SSH) refers to a cryptographic network protocol used in secure communication between devices. SSH encrypts data using cryptographic algorithms, such as Advanced Encryption System (AES) and is often used when logging in remotely to a computer or server.

Secure Shell or SSH simply is a protocol between devices in an encrypted form. Using cryptography, any input we send in a human-readable format is encrypted for travelling over a network -- where it is then unencrypted once it reaches the remote machine, such as in the diagram below.



- SSH allows us to remotely execute commands on another device remotely.
- Any data sent between the devices is encrypted when it is sent over a network such as the Internet

Using SSH to Login to Your Linux Machine:

The syntax to use SSH is very simple. We only need to provide two things:

1. The IP address of the remote machine
2. Correct credentials to a valid account to login with on the remote machine

How to Login Remotely using SSH in System?

- Command: **ssh**
- to login you need Hostname, IP Address & Password of system.
- Syntax: **ssh <username/hostname/systemname>@<IPAddressOfRemoteSystem>**
- Example: **ssh tryhackme@10.81.149.27**
Password: tryhackme

>> Task 3: Introduction to Flags and Switches

A majority of commands allow for arguments to be provided. These arguments are identified by a hyphen and a certain keyword known as flags or switches.

Main Command: **ls** (Using ls to view the contents of a directory)

Argument: **-a** [short for **--all**] (Using ls to view hidden folders/files)

Command: **ls -a**

Argument: **--help** (Listing the options we can use with ls)

Command: **ls --help**

The **Man**(ual) Page:

The manual pages are a great source of information for both system commands and applications available on both a Linux machine, which is accessible on the machine itself and online(<https://linux.die.net/man/>).

To access this documentation, we can use the **man** command and then provide the command we want to read the documentation for. Using our ls example, we would use man ls to view the manual pages for ls like so:

Main Command: **man**

Syntax: **man <command>**

How to Use: **man ls**

Q1) What directional arrow key would we use to navigate down the manual page?

Answer: Down 

Q2) What flag would we use to display the output in a "human-readable" way?

Answer: **-h**

>> Task 4: Filesystem Interaction Continued

In this task, we're going to learn some more commands for interacting with the filesystem to allow us to:

- create files and folders
- move files and folders
- delete files and folders

More specifically, the following commands:

Command	Full Name	Purpose
touch	touch	Create file
mkdir	make directory	Create a folder
cp	copy	Copy a file or folder
mv	move	Move a file or folder
rm	remove	Remove a file or folder
file	file	Determine the type of a file

Protip: Similarly to using cat, we can provide full file paths, i.e. directory1/directory2/note for all of these commands.

Creating Files and Folders (**touch**, **mkdir**):

- Using touch to create a new file

Command: **touch**

Syntax: **touch <FileName>**

Example: **touch Notes**

- Creating a new directory with mkdir

Command: **mkdir**

Syntax: **mkdir <FolderName>**

Example: **mkdir mydirectory**

Removing Files and Folders (**rm**):

rm is extraordinary out of the commands that we've covered so far. You can simply remove files by using **rm**. However, you need to provide the **-R** switch alongside the name of the directory you wish to remove.

- Using **rm** to remove a file

Command: **rm**

Syntax: **rm <FileName>**

Example: **rm Notes**

- Using **rm** recursively to remove a directory

Command: **rm**

Syntax: **rm -R <FolderName>**

Example: **rm -R mydirectory**

Copying and Moving Files and Folders (**cp**, **mv**):

Copying and moving files is an important functionality on a Linux machine. Starting with **cp**, this command takes two arguments:

1. the name of the existing file
2. the name we wish to assign to the new file when copying

cp copies the entire contents of the existing file into the new file.

- Using **cp** to copy a file

Command: **cp**

Syntax: **cp <ExistingFile> <ExistingFile/NewFile>**

Example: **cp Notes Textbook**

- Using **cp** recursively to copy a directory

Command: **cp**

Syntax: **cp -R <ExistingFolder> <ExistingFolder/NewFolder>**

Example: **cp -R mydirectory mydirectory2**

- Using **mv** to move a file

Command: **mv**

Syntax: **mv <ExistingFile> <ExistingFolder/ExistingFile>**

Example: **mv Notes mydirectory**

- Using **mv** to move a folder

Command: **mv**

Syntax: **mv <ExistingFolder> <ExistingFolder>**

Example: **mv mydirectory mydirectory2**

- Using **mv** to rename a file

Command: **mv**

Syntax: **mv <ExistingFile> <NewFile>**

Example: **mv Notes Notes1**

- Using **mv** to rename a folder

Command: **mv**

Syntax: **mv <ExistingFolder> <NewFolder>**

Example: **mydirectory mydirectory3**

Determining File Type:

What is often misleading and often catches people out is making presumptions from files as to what their purpose or contents may be. Files usually have what's known as an extension to make this easier. For example, text files usually have an extension of ".txt". But this is not necessary.

So far, the files we have used in our examples haven't had an extension. Without knowing the context of why the file is there -- we don't really know its purpose. Enter the **file** command. This command takes one argument. For example, we'll use **file** to confirm whether or not the "note" file in our examples is indeed a text file, like so **file Notes**.

- Using **file** to determine the contents of a file

Command: **file**

Syntax: **file <ExistingFile/ExistingFolder>**

Example: **file Notes**

Q1) How would you create the file named "newnote"?

Answer: `touch newnote`

Q2) On the deployable machine, what is the file type of "unknown1" in "tryhackme's" home directory?

Answer: ASCII text

Q3) How would we move the file "myfile" to the directory "myfolder"

Answer: `mv myfile myfolder`

Q4) What are the contents of this file?

Answer: THM{FILESYSTEM}

>> Task 5: Permissions 101

Using **ls -lh** (Long listing format with Human-readable file sizes) or **ls -l** (Long listing format) to list the permissions of all files/folders in the directory.

Main Command: **ls**

Sub Command: **ls -l**

Sub Command: **ls -lh**

Sub Command: **ls -la** (Long listing format with Showing Hidden Files Also)

Briefly: The Differences Between Users & Groups:

The great thing about Linux is that permissions can be so granular, that whilst a user technically owns a file, if the permissions have been set, then a group of users can also have either the same or a different set of permissions to the exact same file without affecting the file owner itself.

Let's put this into a real-world context; the system user that runs a web server must have permissions to read and write files for an effective web application. However, companies such as web hosting companies will have to want to allow their customers to upload their own files for their website without being the webserver system user -- compromising the security of every other customer.

Switching Between Users:

Switching between users on a Linux install is easy work thanks to the **su** command. Unless you are the root user (or using root permissions through sudo), then you are required to know two things to facilitate this transition of user accounts:

The user we wish to switch to

The user's password

The **su** command takes a couple of switches that may be of relevance to you. For example, executing a command once you log in or specifying a specific shell to use. I encourage you to read the man page for **su** to find out more. However, I will cover the **-l** or **--login** switch.

Simply, by providing the **-l** switch to **su**, we start a shell that is much more similar to the actual user logging into the system - we inherit a lot more properties of the new user, i.e., environment variables and the likes.

Command: **su <user>**

Example: `su user2` (keeps your current directory and environment, so you still “see” the previous user’s files.)

Example: `su -l user2` / `su --login user2` (simulates a full login, moves you into user2’s home, so you don’t see the old user’s files unless you navigate back).

Understanding File Permissions in Numeric Format:

In Linux, every file and directory has a set of permissions that control who can read, write, or execute it. These permissions are often displayed in symbolic format, such as:

`rwxrwxrwx`

This format is split into three groups:

Section	Applies To	Example
First 3	Owner	<code>rwx</code>
Next 3	Groups	<code>rwx</code>
Last 3	Others	<code>rwx</code>

Each letter represents a specific permission:

r = read

w = write

x = execute

Converting Symbolic Permissions to Numbers:

Each permission has a numeric value:

Permission	Value
Read (r)	4
Write (w)	2
Execute (x)	1

To calculate the numeric value, we add the values together for each group.

Example: **rw-rw-rw-**

Break it down:

Group	Permissions	Calculation	Value
Owner	rw-	4+2+1	7
Groups	rw-	4+2+1	7
Others	rw-	4+2+1	7

So:

rw-rw-rw- = 777

More Common Examples:

Symbolic	Numeric	Meaning
rw-r-xr-x	755	Owner can do everything, others can read and execute
rw-r--r--	644	Owner can read/write, others can only read
rw------	700	Only the owner has access

Why This Matters:

Understanding numeric permissions is important because:

- Many Linux commands use numeric values (e.g. `chmod 755 file`)
- You can quickly identify security risks
- You can control who can access sensitive files

For example:

`chmod 750 system_overview.txt`

This means:

- Owner: full access
- Group: read + execute
- Others: no access

Q1) On the deployable machine, who is the owner of "important"?

Answer: user2

Q2) What would the command be to switch to the user "user2"?

Answer: **su user2**

Q3) Output the contents of "important", what is the flag?

Answer: THM{SU_USER2}

>> Task 6: Common Directories

♦ /etc

This root directory is one of the most important root directories on your system. The etc folder (short for etcetera) is a commonplace location to store system files that are used by your operating system.

For example, the sudoers file highlighted in the screenshot below contains a list of the users & groups that have permission to run sudo or a set of commands as the root user.

Also highlighted below are the "**passwd**" and "**shadow**" files. These two files are special for Linux as they show how your system stores the passwords for each user in encrypted formatting called sha512.

Command: `cd /etc && ls`

Output: shadow passwd sudoers sudoers.d (Some notable contents of the /etc directory)

♦ /var

The "/var" directory, with "var" being short for variable data, is one of the main root folders found on a Linux install. This folder stores data that is frequently accessed or written by services or applications running on the system. For example, log files from running services and applications are written here (**/var/log**), or other data that is not necessarily associated with a specific user (i.e., databases and the like).

Command: `cd /var && ls`

Output: backups log opt tmp (Some notable contents of the /var directory)

♦ /root

Unlike the **/home** directory, the **/root** folder is actually the home for the "root" system user. There isn't anything more to this folder other than just understanding that this is the home directory for the "root" user. But, it is worth a mention as the logical presumption is that this user would have their data in a directory such as **"/home/root"** by default.

Command: `cd /root && ls`

Output: myfile myfolder passwords.xlsx (Some notable contents of the /root directory)

♦ /tmp

This is a unique root directory found on a Linux install. Short for "**temporary**", the /tmp directory is volatile and is used to store data that is only needed to be accessed once or twice. Similar to the memory on your computer, once the computer is restarted, the contents of this folder are cleared out.

What's useful for us in pentesting is that any user can write to this folder by default. Meaning once we have access to a machine, it serves as a good place to store things like our enumeration scripts.

Command: `cd /tmp && ls`

Output: todelete trash.txt rubbish.bin (Some notable contents of the /tmp directory)

Q1) What is the directory path that would we expect logs to be stored in?

Answer: /var/log

Q2) What root directory is similar to how RAM on a computer works?

Answer: /tmp

Q3) Name the home directory of the root user.

Answer: /root